Verified Settlement Search

BANKs, Formal Self-Description, Controlled IFS Dynamics, Abel–Bellman Navigation, and a Return to Conjectures

Version 1.4

Brian Tenneson
M.A. Mathematics; M.S. Applied Data Science
Currently Unaffiliated

August 22, 2026

Abstract

This manuscript develops a self-contained bridge from verified theorem memory to reflective control and then back to the practical goal of conjecture settlement. It draws together the semi-ideal computer (SIC) and automated theorem prover (ATP) framework of *Depths of Induction and Formalized Self-Awareness*, the BANK/FUTUREBANK and P/R/I/C architecture of *What Checks the Proof?*, the controlled iterated-system and fixed-point viewpoint of DATA 3.0, and the value-function/compass program of AMLD control theory. The main structural result is a BANK–SIC representation theorem: after forgetting record metadata, an AMLD BANK trajectory is a SIC trajectory on an expanded certificate language, and the starting set Γ has the same mathematical function in both frameworks—it is the admitted stage-one base from which certified computation grows. Under explicit certificate-search hypotheses the global P/R/I/C settlement process is therefore an ATP-like target search for a settlement proposition.

The paper then treats the global search as a controlled discrete iterated-function system. Multiple frozen terminal states give a simple obstruction to Hutchinson-style strict contraction on the complete unquotiented state space. In the opposite direction, after identifying all terminal states and restricting to a deterministic finite-settling basin, a weighted path metric makes the induced settlement map a genuine contraction. The remaining hitting time r satisfies $r(Tx) = r(x) - 1$ before settlement; hence $h = -r$ is an exact Abel coordinate, and exponentiation yields a Schröder-type multiplicative coordinate. In the controlled setting the optimal remaining settlement time r^* satisfies Bellman optimality, while $h^* = -r^*$ satisfies a controlled Abel equation. A half-step theorem shows that a compass uniformly accurate to less than $1/2$ on candidate successor states recovers a shortest-settlement action exactly in the unit-cost discrete model.

The Universal Proof-Horizon Operator is then connected explicitly to the SIC and nonstandard-horizon formulations. For an arithmetically represented effective proof presentation, ordinary finite Proof-Horizon halting, standard derivability, and existence of a proof below an unlimited nonstandard cost bound are equivalent on standard targets; the unlimited bound dominates every standard theorem-specific proof horizon without becoming an executable standard runtime. Formal self-description is then given an operational role. A tagged settlement machine can, under effective representability and fair proof search, prove a formal statement of its own ATP status; checked current-state descriptions can be used by the controller without altering verifier semantics. The final sections convert these results into a staged experimental program whose primary endpoint is not prediction accuracy but independently verified settlement of withheld and eventually genuinely difficult conjectures.

Status discipline. Results labeled theorem, proposition, or corollary are proved from the hypotheses stated here. Claims about learned guidance transferring to unseen mathematics are experimental hypotheses unless separately proved. No open mathematical conjecture is claimed settled in this manuscript.

Contents

1 Why another text: from checking proofs to watching search

What Checks the Proof? begins with a deliberately simple slogan:

> "Imagine freely; verify rigidly."

The point is architectural rather than rhetorical. A machine may explore aggressively, hypothesize lemmas, rank branches, or reason about its own search state, but only checker-accepted certificates enter verified mathematical memory [2]. The same text later states the stronger epistemic principle

> "The machine may be uncertain about what to try next while being exact about what it has already certified."

That distinction leaves a natural sequel question. Once verification has been separated from search, what mathematical object is doing the searching, what can it know about itself, and how should that knowledge affect the next move?

The present manuscript is written so that the reader does not need to keep earlier texts open beside it. Definitions needed from the earlier work are restated and connected rather than inserted without explanation. The intellectual line is roughly

$$\begin{aligned}\text{formal system} &\longrightarrow \text{SIC/ATP} \longrightarrow \text{BANKed AMLD}\\ &\longrightarrow \text{controlled IFS} \longrightarrow \text{reflective controller}\\ &\longrightarrow \text{verified settlement.}\end{aligned}$$

The working title *What Watches the Search?* names the reflective-control direction. Two further working titles make the longer trajectory explicit: *Initial Segments of Tegmark's Self-Aware Substructures: Formal Self-Aware Automated Theorem Provers* would isolate the tagged/reflection theory, while *Techniques to Settle Conjectures* would deliberately shift attention back to end-to-end mathematical targets. These are working directions rather than claims that each title already denotes a finished publication. This paper sits between them: it develops enough theory to make "watching the search" useful, but it keeps returning to the question of whether the resulting machine actually reaches a certificate.

Several earlier papers supply pieces of this bridge. *Depths of Induction and Formalized Self-Awareness* formalizes SICs, ATPs, tags, self-theories, and reflection levels [3]. DATA 3.0 represents a verifier-backed conjecture-settling machine by complete state transformations forming a controlled iterated system and transformation semigroup [4]. *Proof Compass Theory* separates learnability, shrinkage, and adaptive control from their unrestricted conjectural forms [5]; the later AMLD compass-control synthesis identifies exact distance-to-settlement with a minimum-time value function, its negative with an Abel coordinate, and Bellman minimization with feedback control [6]. The *Reflective Compass Control Research Program* then proposes checked self-state as input to a safe meta-controller rather than as a new source of proof authority [7]. *Automatic Metalogical Deciders, Part Two* insists that prolonged P/R failure is not itself evidence for independence and uses fair scheduling as the canonical semantics, while treating adaptive scheduling as an experimental comparator [8]. These are not separate metaphors. Under suitable hypotheses they are views of the same machine.

2 The formal starting point and the role of Γ

2.1 Formal systems and consequence

Fix a formal presentation

$$\mathfrak{F} = (\Sigma, A, R),$$

where Σ is a formal language, A is an allowed axiom/hypothesis universe, and R is a declared family of finitary inference rules. For an admitted hypothesis set $\Gamma \subseteq A$, let

$$\mathrm{Con}_{\mathfrak{F}}(\Gamma)$$

denote the least inference-closed set containing Γ. A finite derivation of φ from Γ is written

$$\Gamma \vdash_{\mathfrak{F}} \varphi.$$

The key fact for this paper is not a particular proof calculus. It is that Γ is fixed before search begins and supplies the base against which every derived claim is scoped.

2.2 Semi-ideal computers

The SIC framework makes the evolving theorem state explicit. The earlier definition can be paraphrased as follows [3]. A SIC over a formula domain Y is a pair $M = (C, H)$ with

$$C : \mathcal{P}(Y) \times \mathbb{N}_+ \to \mathcal{P}(Y), \qquad H : \mathcal{P}(Y) \times \mathbb{N}_+ \to \{0, 1\},$$

satisfying, for every $\Gamma \subseteq Y$ and $m \in \mathbb{N}_+$,

$$C(\Gamma, 1) = \Gamma, \tag{1}$$
$$C(\Gamma, m) \subseteq C(\Gamma, m + 1), \tag{2}$$

and, once $H(\Gamma, m) = 1$, both the halting bit and the state persist:

$$H(\Gamma, m + 1) = 1, \tag{3}$$
$$C(\Gamma, m + 1) = C(\Gamma, m). \tag{4}$$

Thus a SIC is not merely an algorithm with an input string. It is a staged computation whose mathematical state begins exactly at Γ and expands monotonically until, if a designated halt occurs, the state freezes.

Depths summarizes its broad ATP notion in a sentence worth retaining:

> "An automated theorem prover ... is a SIC ... intended to enumerate theorems or search for designated targets."

The ellipses suppress the domain qualifiers, not mathematical content [3]. A target-search ATP adds a liveness condition: if the designated target is derivable, some finite stage halts with the target in the materialized state.

2.3 The BANK in an AMLD

An Automated Mathematical Logic Decider (AMLD) begins from a richer environment

$$\mathcal{E} = (\Gamma, c, \mathcal{L}, \mathcal{R}, \mathcal{C}_{\text{check}}, \nu),$$

containing the base Γ, target c, language, inference rules, declared certificate/checker families, and an environment version. Its verified BANK begins with typed records for Γ:

$$\mathcal{B}_0 = \text{seed}(\Gamma, \mathcal{E}).$$

The BANK may then grow by adding checker-accepted derived records. The extra record fields—scope, semantic certificate kind, dependencies, provenance, verifier receipt, environment identifier, and deposit time—matter for safety, but they do not change the mathematical content of an admitted axiom [2].

There is therefore a canonical forgetful map

$$\text{cont} : \{\text{BANK records}\} \to Y^{\dagger},$$

where $Y^{\dagger}$ is an expanded language capable of representing both object-level and declared metalevel certificate statements. For a set of records B, write

$$\text{cont}(B) = \{\text{cont}(b) : b \in B\}.$$

By construction,

$$\text{cont}(\mathcal{B}_0) = \Gamma$$

when the initial metadata are forgotten.

3 The BANK–SIC bridge: settlement is computation with verified banks

The phrase "the BANK begins with Γ" can sound like an implementation convention. In fact it is the direct analogue of the SIC axiom $C(\Gamma, 1) = \Gamma$. The following theorem makes that claim precise.

Definition 3.1 (BANK trajectory)**.** Fix an AMLD instance $(\mathcal{E}, c)$ and write

$$\mathcal{B}_0, \mathcal{B}_1, \mathcal{B}_2, \ldots$$

for its verified BANK states after successive completed atomic turns. Assume:

(B1) **Seed correctness:** $\text{cont}(\mathcal{B}_0) = \Gamma$.

(B2) **Monotone deposit:** $\mathcal{B}_m \subseteq \mathcal{B}_{m+1}$ until terminalization.

(B3) **Verifier gating:** every new mathematical record in $\mathcal{B}_{m+1} \setminus \mathcal{B}_m$ is admitted by the declared checker relative to its recorded scope and dependencies.

(B4) **Terminal freezing:** once a terminal settlement certificate is accepted, later BANK states are unchanged.

Let $H_D(\Gamma, m+1) = 1$ exactly when the AMLD has terminalized by BANK stage m.

Theorem 3.2 (BANK–SIC representation)**.** *Under (B1)–(B4), define for $m \in \mathbb{N}_+$*

$$C_D(\Gamma, m) := \mathrm{cont}(\mathcal{B}_{m-1}).$$

Then $M_D = (C_D, H_D)$ is a SIC over the expanded declared certificate language $Y^\dagger$.

Proof. By (B1),

$$C_D(\Gamma, 1) = \mathrm{cont}(\mathcal{B}_0) = \Gamma,$$

which is the SIC seed axiom. By (B2), $\mathcal{B}_{m-1} \subseteq \mathcal{B}_m$, hence

$$C_D(\Gamma, m) \subseteq C_D(\Gamma, m+1).$$

If $H_D(\Gamma, m) = 1$, terminalization has already occurred. By definition the halting condition remains true at every later stage, and by (B4) the BANK freezes. Therefore

$$H_D(\Gamma, m+1) = 1, \qquad C_D(\Gamma, m+1) = C_D(\Gamma, m).$$

These are exactly the SIC persistence and freezing axioms. The verifier-gating condition (B3) is stronger than the bare SIC axioms: it supplies the intended semantic reason that BANK growth represents certified mathematical computation rather than arbitrary set growth. □

Corollary 3.3 (Γ has the same structural function)**.** *Modulo BANK metadata, Γ plays the same mathematical role in the SIC and AMLD frameworks: it is the admitted initial state from which the certified computational state grows, and it remains the reference base for scope and consequence throughout the run.*

Proof. In the SIC, the role is forced by $C(\Gamma, 1) = \Gamma$. In the AMLD, Theorem 3.2 identifies the metadata-forgotten seed BANK with the same stage-one state. Every later certified state extends that base. Thus the difference is representational—plain formulas versus typed records—not a difference in the logical function of Γ. □

This is the sense in which settlement is *computing with banks*. A SIC materializes a monotone theorem state. An AMLD materializes a richer, provenance-aware verified state. The latter can be viewed as a typed lift of the former.

Remark 3.4 (The AMLD BANK is strictly richer data). The corollary does not identify a BANK record with a formula. An AMLD record may carry object/metalogical level, certificate body, dependencies, role provenance, scope, and checker receipt. The claim is that after applying cont, the underlying admitted mathematical state has the SIC form. This is why BANK memory can safely support cross-role reuse without confusing discovery history with logical consequence.

3.1 From SIC to ATP-like settlement search

Let the declared terminal certificate kinds be

$$\mathsf{P}, \quad \mathsf{R}, \quad \mathsf{I}, \quad \mathsf{C},$$

standing respectively for a checked proof of c, a checked proof of $\neg c$, an accepted independence metacertificate, and an accepted inconsistency certificate for the declared base. In an expanded metatheory, introduce the settlement proposition

$$\mathsf{Settled}_{\Gamma,c} := \mathsf{P}_{\Gamma,c} \vee \mathsf{R}_{\Gamma,c} \vee \mathsf{I}_{\Gamma,c} \vee \mathsf{C}_{\Gamma}.$$

Theorem 3.5 (Conditional settlement-ATP theorem)**.** *Assume Theorem 3.2. Suppose in addition that:*

(S1) every accepted terminal certificate causes a verified BANK record entailing $\mathsf{Settled}_{\Gamma,c}$*;*

(S2) the declared terminal certificate families are effectively enumerable/checkable on the instance class under consideration; and

(S3) the combined P/R/I/C schedule contains a persistent fair fallback that eventually examines every candidate in those declared families.

Then M_D *is a target-search ATP for* $\mathsf{Settled}_{\Gamma,c}$ *relative to that declared settlement language: whenever an admissible terminal certificate exists, the combined search eventually halts with a verified settlement record.*

Proof. Theorem 3.2 gives the SIC structure. By (S1), halting implies that the designated settlement proposition is represented in the materialized state. If an admissible terminal certificate exists, effective enumerability/checkability and fair persistent coverage imply that the relevant candidate is eventually examined. When examined, the checker accepts it, it is deposited, and terminalization occurs. This is exactly the target-search liveness condition for the declared settlement proposition. □

Caution 3.6 (No universal decision procedure)*.* The theorem is deliberately conditional. In arbitrary sufficiently expressive theories, independence or other metacertificate families need not be available as one recursively searchable complete class, and no scheduling trick removes undecidability. The result says that where a certificate class is effectively covered, reflective or heuristic search can be placed around it without destroying the covered guarantee.

4 The whole P/R/I/C schedule as one machine

A common notation writes the role schedule locally as

$$P(1), R(1), I(1), C(1), P(2), R(2), I(2), C(2), \ldots.$$

The parenthesized number can mean the local turn count of the role. If a single global clock is preferred, the same run can be written

$$P[1], R[2], I[3], C[4], P[5], R[6], \ldots.$$

The distinction is bookkeeping. The important point is that the schedule is not four unrelated theorem provers observed from outside. Once BANK, verifier history, resource state, FUTUREBANK, and controller state are shared, the schedule defines one evolving machine.

A convenient complete state is

$$\Sigma_m = (\mathcal{B}_m, \mathcal{FB}_m, \mathsf{H}_m, \mathsf{Ref}_m, \mathsf{Ctrl}_m, q_m),$$

where q_m includes whatever phase/scheduler data are needed to determine the next role. DATA 3.0 uses a more detailed complete state containing the theory/target/metatheory triple, typed BANK, multiple search frontiers, untrusted proposal buffers, history, learned parameters, resource allocation, certificate records, status, and a round counter [4]. The mathematical requirement is Markovian completeness: the next update must be determined by the declared state together with any frozen external policy.

5 The Universal Proof-Horizon Operator and the NSA unlimited horizon

The preceding SIC discussion is about staged computation. A closely related earlier construction, the *Universal Proof-Horizon Operator*, starts from an effective presentation of proofs and asks a more concrete question: if a target is provable, how far out in an effectively ordered proof-cost search must one go before a proof appears? [1] The nonstandard-analysis chapter of *What Checks the Proof?* asks a complementary idealized question: what happens if the bound is not merely very large but larger than every standard natural? [2]

These two uses of the word *horizon* should not be conflated. The Proof-Horizon paper writes its proof-cost function as $c(p)$ and its operator as $H_{T,V,c}$. In the present manuscript, c already denotes the conjecture under settlement, so this section renames the proof-cost function κ and writes the operator as

$$\mathcal{H}_{T,V,\kappa}.$$

Likewise, the nonstandard chapter writes its unlimited bound as H. To prevent that symbol from being mistaken for the Proof-Horizon operator, this section denotes the unlimited nonstandard natural by K.

5.1 The ordinary computable horizon

An *effective proof presentation* consists, schematically, of a formal theory T, a total mechanical verifier V, and a total computable cost function

$$\kappa : \mathbb{N} \to \mathbb{N}$$

with effectively finite bounded sublevel sets: for every standard b, the set $\{p : \kappa(p) \leq b\}$ is finite and effectively listable. This is the effective-properness condition used in the Proof-Horizon paper. Let

$$\sigma_0, \sigma_1, \sigma_2, \ldots$$

be a fixed computable enumeration of the sentences of the language.

The Universal Proof-Horizon theorem gives a partial computable operator $\mathcal{H}_{T,V,\kappa}(n)$ that searches the cost layers in increasing order. It halts exactly when $T \vdash \sigma_n$ and, when it halts, returns the fixed tie-broken least proof code among proofs of minimum κ-cost [1]. Define the corresponding theorem-specific finite horizon by

$$h_{T,V,\kappa}(n) := \kappa(\mathcal{H}_{T,V,\kappa}(n))$$

whenever the operator is defined. Thus $h_{T,V,\kappa}$ is partial computable on exactly the theorem indices.

This is a semidecision boundary, not a decision procedure. If σ_n is not a theorem, ordinary search need not halt. The fact that a verifier can recognize a proof when one arrives does not provide an ordinary finite test for the absence of every proof.

5.2 The nonstandard bounded proof condition

The NSA formulation needs a stronger semantic environment than an arbitrary nonstandard model of arithmetic. Expand the first-order language of the standard natural-number structure by symbols interpreting the verifier relation V and the total cost function κ (equivalently, use formulas defining their graphs), and call the resulting standard structure $\mathcal{N}_{T,V,\kappa}$. Let

$$\mathcal{N}_{T,V,\kappa} \preccurlyeq M$$

be an elementary extension. Let $K \in M$ be *unlimited*, meaning

$$K > m \qquad \text{for every standard } m \in \mathbb{N}.$$

For a standard sentence φ, define the cost-bounded proof-existence condition

$$B^M_{K,\kappa}(\varphi) \quad :\Longleftrightarrow \quad M \models (\exists p)\big[\kappa^M(p) \le K \ \wedge \ V^M(p, \ulcorner\varphi\urcorner) = 1\big]. \tag{5}$$

This is a truth condition in the elementary extension. It is not an instruction to an ordinary computer to execute K steps.

Remark 5.1 (Arithmetic extension versus full nonstandard analysis). Theorem 5.2 uses only an elementary extension of the expanded natural-number structure. A full nonstandard-analysis universe with a star map, internal sets, and hyperfinite intervals is not required. If such an enlargement is chosen, K may be viewed as an unlimited hypernatural and the bounded search may be pictured as an internal hyperfinite scan; the theorem itself depends only on elementarity and unlimitedness. This is the same framework separation enforced in the nonstandard chapter of *What Checks the Proof?* [2].

The $B^M_H(\varphi)$ notation used in the nonstandard chapter of *What Checks the Proof?* is the corresponding proof-code-bounded special case

$$M \models (\exists p)\big[p \le H \wedge \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner\varphi\urcorner)\big].$$

In effect, that special case orders witnesses by the code-value bound itself. Equation (5) is the version aligned with the Proof-Horizon paper's more general proof-cost function.

Theorem 5.2 (Proof-Horizon–NSA equivalence)**.** *Under the hypotheses above, for every standard sentence σ_n,*

$$\boxed{\mathcal{H}_{T,V,\kappa}(n)\downarrow \quad\Longleftrightarrow\quad T \vdash \sigma_n \quad\Longleftrightarrow\quad B^M_{K,\kappa}(\sigma_n).}$$

Whenever these equivalent conditions hold,

$$h_{T,V,\kappa}(n) < K.$$

Proof. The first equivalence is the Universal Proof-Horizon theorem: the operator halts exactly on theorem indices.

Assume $T \vdash \sigma_n$. Then there is an ordinary finite proof code p_* accepted by V. Its cost $\kappa(p_*)$ is a standard natural. Since K exceeds every standard natural,

$$\kappa(p_*) < K.$$

The elementary extension agrees with the standard structure on the represented verifier and cost relations for the standard parameters, so the same standard witness p_* establishes $B^M_{K,\kappa}(\sigma_n)$.

Conversely, suppose $B^M_{K,\kappa}(\sigma_n)$ holds. Dropping the bound yields

$$M \models (\exists p)\, V^M(p, \ulcorner\sigma_n\urcorner) = 1.$$

Because the target code is standard and M is an elementary extension of the standard expanded structure,

$$\mathcal{N}_{T,V,\kappa} \models (\exists p)\, V(p, \ulcorner\sigma_n\urcorner) = 1.$$

Thus an ordinary finite T-proof of σ_n exists, so $T \vdash \sigma_n$.

Finally, when the horizon operator halts, its minimum cost $h_{T,V,\kappa}(n)$ is a standard natural, and therefore is smaller than the unlimited K. □

Corollary 5.3 (Unlimited bounds dominate all standard theorem horizons)**.** *For every standard theorem index n,*

$$h_{T,V,\kappa}(n) < K.$$

Thus one unlimited nonstandard cost bound lies beyond every theorem-specific standard finite Proof Horizon, although it does not supply an executable universal standard cutoff.

This is the precise sense in which the NSA construction extends the horizon idea. The ordinary operator moves through

$$0, 1, 2, \ldots, h_{T,V,\kappa}(n)$$

until the first minimum-cost proof layer succeeds. The NSA statement places an idealized bound K beyond every standard finite layer at once. It does not replace the theorem-specific minimum: $h_{T,V,\kappa}(n)$ remains the exact finite horizon for a provable target, while K is an external upper bound dominating all standard such horizons.

5.3 SIC stages, AMLD routes, and the same boundary

When κ is proof length and the canonical breadth-first SIC materializes exactly the formulas reachable within successive proof-length layers, its first halting stage on a theorem is the shortest proof length. Thus the SIC first-halting stage and the corresponding Proof Horizon are two descriptions of the same finite boundary in that cost model. More general effectively proper costs give analogous cost layers even when they are not identical to SIC stage count.

For conjecture settlement, the generalized bounded condition can be applied to the positive object-level channels

$$B^M_{K,\kappa}(c), \qquad B^M_{K,\kappa}(\neg c), \qquad B^M_{K,\kappa}(\bot).$$

They correspond, respectively, to the existence of standard finite P-, R-, and C-certificates covered by the represented verifier. If T is consistent, then the simultaneous failures

$$\neg B^M_{K,\kappa}(c) \quad \text{and} \quad \neg B^M_{K,\kappa}(\neg c)$$

are equivalent, at the metatheoretic level, to

$$T \nvdash c \quad \text{and} \quad T \nvdash \neg c.$$

This is a derivability-based independence classification. It must not be confused with an ordinary finite I-certificate produced by a standard AMLD. If the architecture requires finite model, relative-consistency, or other metacertificates for I, those remain separate certificate obligations. If a contradiction certificate exists, the C channel should receive the declared contradiction-dominance priority.

Caution 5.4 (The NSA condition is not a new standard decider)*.* The equivalence

$$\neg B^M_{K,\kappa}(\varphi) \quad \Longleftrightarrow \quad T \nvdash \varphi$$

is a semantic consequence of the chosen elementary extension on standard targets. It does not give a standard computer an effective test for the divergence of $\mathcal{H}_{T,V,\kappa}(n)$. Compactness, elementarity, and unlimitedness provide a model-theoretic idealization, not an ordinary finite runtime. This preserves the Church–Gödel boundary of the Proof-Horizon construction.

The resulting lineage is therefore

$$\boxed{\text{Proof Horizon} \longrightarrow \text{SIC horizon} \longrightarrow \text{NSA unlimited horizon} \longrightarrow \text{P/R/I/C settlement semantics.}}$$

The first two are ordinary staged/computable search notions under their stated hypotheses. The third is a nonstandard semantic idealization. DATA 3.0 then supplies the dynamical question that remains after the horizon is characterized: how should the evolving complete state be controlled so that an actual verifier-gated run reaches the relevant terminal fixed-point class efficiently?

6 DATA 3.0: the search is a controlled discrete iterated system

DATA 3.0 states the central point explicitly:

> "Data 3.0 is more accurately a controlled or place-dependent iterated system than a classical contractive Hutchinson IFS."

This sentence is important because it separates two claims that are often conflated [4]. The first is structural: the machine evolves by repeatedly selecting state maps. The second is metric: those maps might or might not be contractions. The structural claim is immediate once complete state is fixed; the contraction claim requires a separate theorem and, on the unquotiented state space, is generally false.

Let X be the complete state space. For each active subsystem or role j, let

$$F_j : X \to X$$

be the realized verified state transformation. The map can include proposal, verification, BANK/history update, and controller bookkeeping. A deterministic controller π gives

$$j_n = \pi(x_n), \qquad x_{n+1} = F_{j_n}(x_n).$$

Finite compositions generate a transformation semigroup

$$S = \langle F_j : j \in J \rangle \subseteq X^X.$$

The semigroup need not commute: changing the order of two search/control transformations generally changes which BANK and history the second one sees.

Theorem 6.1 (Controlled discrete IFS representation)**.** *Any deterministic AMLD implementation whose complete state contains every variable needed for the next update and whose admissible atomic subsystems act by maps $F_j : X \to X$ is a discrete-time controlled/place-dependent iterated system. Under a fixed deterministic controller π, its orbit is the iteration of the state-dependent map family according to $j_n = \pi(x_n)$.*

Proof. At each discrete time n, the controller selects one index j_n from the declared family and the next complete state is $F_{j_n}(x_n)$. This is exactly iteration of a family of state transformations with place-dependent control. No contractivity or fractal-attractor hypothesis is needed for this representation. □

6.1 Round robin can be made autonomous

For a strict four-role cyclic schedule, augment the state by

$$q \in \mathbb{Z}/4\mathbb{Z}$$

and define $a(0) = P$, $a(1) = R$, $a(2) = I$, $a(3) = C$. On

$$\widetilde{X} = X \times \mathbb{Z}_4$$

define

$$T(x, q) = \big(F_{a(q)}(x), q + 1 \pmod 4\big).$$

Then iterating a single map T reproduces the cyclic P/R/I/C schedule. One full round may alternatively be packaged as a macro-update

$$T_4 = \text{Terminalize} \circ F_C \circ F_I \circ F_R \circ F_P,$$

with terminal checks inserted after atomic actions when immediate freezing is required.

This conversion is useful later: controlled search can be studied either as a semigroup of available maps or, after freezing a policy/schedule, as an autonomous discrete dynamical system.

7 Settlement as fixed-point geometry

Let $\mathcal{C} \subseteq X$ be the set of states containing an accepted terminal certificate and terminal status. The DATA 3.0 construction uses terminalization so that accepted settlement states are absorbing. Under a macro-update T, its fixed-point theorem takes the form

$$\text{Fix}(T) = \mathcal{C}$$

when every nonterminal macro-round changes a monotone counter and every terminal state thereafter remains unchanged [4]. The counter rules out nonterminal cycles in the complete state.

The fixed-point identity is a stopping-state theorem, not a reachability theorem. It says what settlement looks like if it happens. It does not say that every orbit reaches settlement. This distinction will matter throughout the paper.

8 A refutation: the complete AMLD is not a Hutchinson contractive IFS

Hutchinson's classical IFS framework uses a finite family of contractions on a complete metric space [12]. DATA 3.0 deliberately did not claim this stronger property. The terminal geometry now yields a direct obstruction.

Theorem 8.1 (Full-state noncontractivity)**.** *Let (X, d) be any metric realization of the complete AMLD state space. Suppose two distinct certified terminal states $x_*, y_* \in \mathcal{C}$ are left unchanged by every subsystem map F_j. Then no F_j is a strict contraction on X. Consequently the family $\{F_j\}$ is not a Hutchinson-style contractive IFS on the complete unquotiented state space.*

Proof. Fix j. Since terminal states freeze,

$$F_j(x_*) = x_* \qquad \text{and} \qquad F_j(y_*) = y_*.$$

If F_j were a strict contraction with constant $0 \le q < 1$, then

$$d(x_*, y_*) = d(F_j x_*, F_j y_*) \le q\, d(x_*, y_*).$$

Because $x_* \neq y_*$, the metric gives $d(x_*, y_*) > 0$. Division yields $1 \le q$, contradiction. □

Corollary 8.2 (Macro-map obstruction)**.** *If a deterministic settlement macro-map T has at least two distinct terminal fixed states, then T cannot be a strict contraction under any genuine metric on the complete unquotiented state space.*

The theorem is stronger than a failed search for the right metric. As long as different terminal states remain metrically distinct and are all fixed, no metric can make a shared terminal-freezing map strictly contractive.

Globally, a multi-target architecture naturally has many distinct terminal states because the target is part of complete state. Within a single target fiber there may also be multiple frozen terminal states corresponding to different accepted certificates or histories, unless terminalization deliberately canonicalizes them into one state.

9 A complementary theorem: terminal quotienting can create a contraction

The noncontractivity theorem suggests a natural repair: identify all terminal states. Surprisingly, on a deterministic basin in which every state settles after finitely many steps, this repair is enough to construct an explicit contraction metric.

Definition 9.1 (Finite-settling basin)**.** For a deterministic map $T : X \to X$ with terminal fixed set $\mathcal{C}$, define

$$B := \{x \in X : \exists n \ge 0 \text{ with } T^n x \in \mathcal{C}\}.$$

For $x \in B$, define the remaining hitting time

$$r(x) := \min\{n \ge 0 : T^n x \in \mathcal{C}\}.$$

Thus $r(x) = 0$ exactly on $\mathcal{C}$.

Identify all terminal states and no others:

$$x \sim y \quad \Longleftrightarrow \quad x = y \text{ or } (x, y \in \mathcal{C}).$$

Let

$$\bar{B} = B/\!\sim$$

and denote the common terminal class by $*$. Since T fixes $\mathcal{C}$, it induces a well-defined map

$$\bar{T} : \bar{B} \to \bar{B}.$$

For a nonterminal class $\bar{x}$ define $r(\bar{x}) = r(x)$ and set $r(*) = 0$.

Lemma 9.2 (Depth drops by one)**.** *If $\bar{x} \neq *$, then*

$$r(\bar{T}\bar{x}) = r(\bar{x}) - 1.$$

Proof. If $r(\bar{x}) = m > 0$, then $T^m x$ is terminal and no earlier iterate is. Starting from Tx, the first terminal iterate therefore occurs after exactly $m - 1$ further steps. □

Theorem 9.3 (Quotient contractivization)**.** *Fix any $q \in (0, 1)$. On $\bar{B}$, join every nonterminal vertex v to its parent $\bar{T}v$. Give that edge length*

$$w(v, \bar{T}v) := q^{-r(v)}.$$

Let d_q be the induced path metric. Then $(\bar{B}, d_q)$ is a complete metric space and

$$d_q(\bar{T}x, \bar{T}y) \leq q\, d_q(x, y)$$

for all $x, y \in \bar{B}$. Hence $\bar{T}$ is a strict contraction with unique fixed point $*$.

Proof. Every vertex has a unique parent and a finite directed path to $*$. The underlying undirected graph is connected. It is acyclic: if an undirected cycle existed, choose a vertex of maximal depth r on the cycle. Both cycle-neighbors would have to lie one level closer to $*$, but a vertex has only one parent, contradiction. Thus the graph is a tree and the path metric is well-defined.

Consider an edge from a vertex v of depth $k = r(v)$ to its parent. If $k = 1$, both edge endpoints map to $*$, so the image length is 0. If $k \geq 2$, the image is the parent edge at depth $k - 1$, whose weight is

$$q^{-(k-1)} = q\, q^{-k}.$$

Therefore applying $\bar{T}$ to any path produces a walk whose total length is at most q times the original path length. Taking the shortest path between x and y gives

$$d_q(\bar{T}x, \bar{T}y) \leq q\, d_q(x, y).$$

Finally, every nonzero edge has length at least $q^{-1} > 0$. Hence every Cauchy sequence is eventually constant, so the metric is complete. A strict contraction on a complete metric space has a unique fixed point; here that point is the collapsed terminal class $*$. □

Remark 9.4 (What this does and does not contract). Theorem 9.3 applies to a frozen deterministic settlement map on its finite-settling basin after terminal quotienting. It does not imply that every individual control map F_j in the original multi-map system is contractive under one common metric. Thus it does not undo Theorem 8.1; it shows that the obstruction is partly representational and disappears after quotienting the very multiplicity that caused it.

This pair of theorems gives a useful two-level picture:

complete verifier-preserving state: noncontractive

while

terminal-quotiented deterministic settling basin: contractivizable.

10 Abel and Schröder coordinates arise from settlement time

Lemma 9.2 already has the form of a functional equation. The conventional Abel equation for iteration is

$$h(Tx) = h(x) + 1.$$

Define

$$h(x) := -r(x)$$

on nonterminal points of the finite-settling basin. Then

$$h(Tx) = -r(Tx) = -(r(x) - 1) = h(x) + 1.$$

Thus the remaining settlement time is not merely analogous to an Abel coordinate; its negative is an exact Abel coordinate before the absorbing boundary.

Proposition 10.1 (Hitting-time Abel coordinate)**.** *On any deterministic finite-settling basin,*

$$h = -r$$

satisfies

$$h(Tx) = h(x) + 1$$

for every nonterminal state x.

The terminal state itself cannot carry a finite Abel translation coordinate: if $Tx_* = x_*$, then Abel's equation would demand $h(x_*) = h(x_*) + 1$. The coordinate therefore belongs naturally to the pre-settlement basin and terminates at an absorbing boundary, exactly as emphasized in DATA 3.0 [4].

Exponentiation turns translation into multiplication. For $q \in (0, 1)$ define

$$\psi(x) = q^{-r(x)} = q^{h(x)}.$$

Then before settlement,

$$\psi(Tx) = q^{-(r(x)-1)} = q\, q^{-r(x)} = q\psi(x).$$

This is a Schröder-type equation. The same factor q that appears in the quotient contraction metric therefore appears as the multiplicative eigenvalue of the exponentiated Abel coordinate. The edge weight in Theorem 9.3 is precisely $\psi(v)$ at the deeper endpoint.

The chain is consequently exact:

$$\boxed{\text{remaining settlement time} \to \text{Abel translation} \to \text{Schröder scaling} \to \text{contraction metric}}.$$

11 Control changes the question: the optimal Abel–Bellman theorem

The retrospective coordinate $r(x)$ depends on an already-fixed policy. Conjecture settlement needs a prospective question: among the moves available now, which move minimizes the remaining cost to a certificate?

Let $A(x)$ be the finite or countable set of admissible control actions at a nonterminal state x. An action may mean giving the next quantum to P, R, I, or C; selecting a theorem-search tunnel; proving

a reusable lemma; invoking a representation change; or choosing among other verifier-neutral search transformations. For simplicity first give every action unit cost and write

$$F_a(x)$$

for the successor state.

Definition 11.1 (Optimal remaining settlement time)**.** On states from which verified settlement is reachable, define

$$r^*(x) = \min_{\pi}\{\text{number of further controlled steps needed by policy } \pi \text{ to reach } \mathcal{C}\}.$$

Set $r^*(x) = 0$ on $\mathcal{C}$ and define

$$h^*(x) := -r^*(x).$$

Theorem 11.2 (Controlled Abel–Bellman settlement theorem)**.** *For every nonterminal state with finite r^*,*

$$r^*(x) = 1 + \min_{a \in A(x)} r^*(F_a x),$$

and equivalently

$$\boxed{\max_{a \in A(x)} h^*(F_a x) = h^*(x) + 1.}$$

Any action

$$a^*(x) \in \arg\max_{a \in A(x)} h^*(F_a x)$$

is a shortest-settlement action.

Proof. Any successful policy from x must take one admissible first action and then complete settlement from the successor. Therefore optimality has the Bellman decomposition

$$r^*(x) = 1 + \min_a r^*(F_a x).$$

Negating and using $-\min u_a = \max(-u_a)$ yields

$$h^*(x) = -1 + \max_a h^*(F_a x),$$

which is the displayed controlled Abel equation. An action attaining the maximum attains the minimum successor hitting time and is therefore shortest-settlement. □

This theorem unifies three vocabularies. In dynamic programming, r^* is a unit-cost minimum-time value function [13]. In proof-ocean language, it is exact distance-to-settlement. In iteration theory, $-r^*$ is the coordinate in which an optimal action advances translation by exactly one. AMLD compass-control integration had already identified these objects in deterministic proof DAGs; the present statement makes the controlled Abel equation explicit [6].

12 A discrete robustness theorem: the half-step compass guarantee

An exact h^* is usually as difficult to compute as the remaining search itself. The practical question is whether an approximation can still choose the correct next move. Discreteness supplies a sharp answer.

Theorem 12.1 (Half-step compass theorem)**.** *Assume the unit-cost model of Theorem 11.2. Let $\widehat{h}$ be an approximate compass. Suppose that at every state encountered, for every candidate successor $y = F_a x$,*

$$|\widehat{h}(y) - h^*(y)| < \frac{1}{2}.$$

Then any greedy controller

$$\widehat{\pi}(x) \in \arg\max_{a \in A(x)} \widehat{h}(F_a x)$$

selects a shortest-settlement action. Starting from x_0, it reaches settlement in exactly $r^*(x_0)$ steps.

Proof. Let x be nonterminal. An optimal successor y^* satisfies

$$r^*(y^*) = r^*(x) - 1,$$

so

$$h^*(y^*) = h^*(x) + 1.$$

Any nonoptimal successor z cannot have the same remaining depth, because then it would also be optimal. Since r^* is integer-valued,

$$r^*(z) \geq r^*(x),$$

hence

$$h^*(z) \leq h^*(x).$$

Thus the true compass gap between any optimal successor and any nonoptimal successor is at least 1. The strict error bound gives

$$\widehat{h}(y^*) > h^*(x) + \frac{1}{2}$$

while

$$\widehat{h}(z) < h^*(x) + \frac{1}{2}.$$

Therefore no nonoptimal successor can tie or outrank all optimal successors under $\widehat{h}$. The greedy rule chooses an optimal action. Repeating the argument at each subsequent state gives a trajectory of optimal remaining depth, so settlement occurs after exactly $r^*(x_0)$ steps. □

The threshold $1/2$ is not a tuning parameter. It comes from the unit spacing of integer settlement depths. With unequal action costs, the analogous condition is an action-gap condition: prediction error must be less than half the true separation between the best and second-best action values.

Corollary 12.2 (Local ranking, not global perfection)**.** *In the unit-cost model, a learned compass does not need globally exact values to reproduce shortest-settlement control. It is sufficient to preserve the ordering between optimal and nonoptimal immediate successors; the uniform $< 1/2$ condition is one simple sufficient guarantee.*

This matters experimentally. Mean squared error on all states can be a secondary diagnostic. The operational quantity is whether the approximation preserves the next-action ordering often enough to reduce verified time-to-settlement.

13 BANKs are part of the control state, not just proof storage

A verified lemma can be logically conservative and computationally decisive. If L is already derivable from Γ, adding a checked record of L to the BANK does not strengthen the consequence relation:

$$\mathrm{Con}(\Gamma \cup \{L\}) = \mathrm{Con}(\Gamma).$$

But it can change the operational state because a search process no longer needs to rediscover L.

This is why the value state should be written schematically as

$$V^*(s, \mathcal{B}, \rho, \ldots),$$

not merely $V^*(s)$. The earlier AMLD control synthesis makes this point directly: a theorem shell can alter the future geometry of search even when it does not strengthen the underlying theory [6]. *Automatic Metalogical Deciders, Part Two* similarly distinguishes final-proof membership from downstream BANK value: a lemma absent from one final certificate may still save substantial work on later P/R/I routes [8].

Definition 13.1 (Marginal BANK value)**.** For a verified candidate lemma L, define its state-relative operational value under a chosen cost model by

$$\Delta_L(x) := r^*(x) - r^*(x \oplus L),$$

where $x \oplus L$ denotes the same complete state with a verifier-accepted reusable record for L added to the admissible BANK.

This quantity need not be positive in every implementation: storing or indexing a useless lemma can incur overhead. But the definition states the correct question. A lemma is valuable when its verified availability collapses future settlement cost, not merely because it is elegant or because it appeared in a previous final proof.

14 Formal self-description: what the machine can prove about itself

14.1 From Tegmark's SAS idea to tagged ATPs

Max Tegmark's discussion of self-aware substructures (SASs) is cosmological, not an ATP specification. The earlier self-awareness paper quotes his implicit criterion:

> "a substructure is self-aware if it thinks that it is self-aware"

and interprets only the formal shape of self-ascription, not subjective consciousness [11, 3]. The theorem-proving specialization replaces phenomenology with a tag and formal statements about the tagged machine.

Let D be the global settlement machine and let

$$\langle D \rangle$$

be its designated code/name in a declared metatheory T. A formula $\varphi(\langle D \rangle)$ is self-descriptive when its intended subject is that tag. Define

$$\mathrm{Self}(D) = \{\varphi(\langle D \rangle) : D \vdash_T \varphi(\langle D \rangle)\}.$$

A minimal Level-1 statement in the *Depths* sense is

$$D \vdash_T \mathrm{ATP}(\langle D \rangle).$$

The point is not that a symbol can refer to itself by magic. The specification, coding, metatheory, and proof checker must make the assertion a genuine formal theorem.

14.2 Can the settlement machine prove that it is an ATP?

The external theorem has already been proved conditionally: Theorem 3.5 gives precise hypotheses under which the global BANKed settlement machine is a target-search ATP. Turning that external proof into the machine's own self-theory requires an internalization step.

Theorem 14.1 (Level-1 self-description capability)**.** *Let D be a global settlement machine satisfying Theorem 3.5. Let T be an effectively axiomatized metatheory capable of representing the frozen specification of D, its tag $\langle D \rangle$, and the predicate* $\mathrm{ATP}(x)$*. Assume:*

(R1) there is a finite T-proof, from checker-admitted specification facts S_D, of $\mathrm{ATP}(\langle D \rangle)$*;*

(R2) the relevant proof-search component of D fairly enumerates T-proof candidates while preserving progress; and

(R3) the intrinsic verifier accepts exactly the valid declared T-proof certificates for this self-description channel.

Then, after finitely many effective self-description search steps,

$$D \vdash_T \mathrm{ATP}(\langle D \rangle).$$

Hence D has Level-1 formal self-awareness in the tagged-ATP sense of Depths*.*

Proof. By (R1), a finite valid certificate exists. By fair effective enumeration with persistent progress, (R2) guarantees that the certificate is eventually proposed. By (R3), the verifier accepts it and the corresponding self-descriptive theorem enters the verified state. Therefore D proves the displayed formula about its own tag. □

Caution 14.2 (Capability is not yet a concrete implementation certificate)*.* The theorem gives a constructive route to Level 1, but an actual software release should not be called Level 1 merely because the mathematical architecture could support such a proof. The concrete tagged specification and checker-accepted proof of $\mathrm{ATP}(\langle D \rangle)$ should be produced and archived.

14.3 Reflection depth and checked state-awareness are different

Depths defines a finite reflection sequence

$$\theta_0 = \mathrm{ATP}(\langle D \rangle), \qquad \theta_{k+1} = \mathrm{Pr}(\langle D \rangle, \langle \theta_k \rangle),$$

and grades reflection by which terms the machine proves [3]. *What Checks the Proof?*, however, deliberately uses a different architectural scale for checked self-description. Its Level 2 includes verified dynamic facts such as the current BANK identifier, FUTUREBANK dependencies, verifier-history summaries, resource counters, and fallback status [2].

These measure different things. Iterating a provability predicate can add formal reflection depth without telling a controller which route is currently starved. Conversely, a controller can possess precise checked state facts without proving a long tower of nested statements about its own provability.

It is therefore useful to write a two-coordinate descriptor

$$\mathrm{SA}(D) = \big(\mathrm{lev}_{\mathrm{ref}}(D), \mathrm{lev}_{\mathrm{state}}(D)\big),$$

where the first coordinate refers to the declared reflection schema and the second to checked architectural state description. This notation is a proposal for keeping the two ideas separate, not a claim that either coordinate measures consciousness.

15 Reflection can steer search without changing proof

Let the complete reflective state be

$$x = (\mathcal{B}, \mathcal{F}B, \mathsf{H}, \mathsf{Ref}, \mathsf{Ctrl}, \ldots).$$

The selector may use all coordinates. The verifier need not. If intrinsic acceptance is a function only of the frozen environment, admissible BANK dependencies, and the semantic certificate body, then changing reflection or controller state cannot change whether a fixed certificate is valid. This is the verification-neutrality principle of *What Checks the Proof?* [2].

Theorem 15.1 (Reflective extension preserves controlled-system form)**.** *Suppose $(X, \{F_j\}, \pi)$ is a controlled discrete iterated system for an AMLD. Let R be a space of checked or checkable reflective-state records, and suppose each extended subsystem has a deterministic/effective update*

$$\widehat{F}_j : X \times R \to X \times R.$$

Then the reflective machine is again a controlled discrete iterated system on $\widehat{X} = X \times R$. If verifier acceptance remains independent of the reflective coordinate for fixed semantic certificate input, the reflective extension changes search policy without changing certificate semantics.

Proof. Closure under the maps $\widehat{F}_j$ gives the controlled iterated-system representation immediately. Verifier neutrality follows from the stated interface independence: a coordinate that is not an argument of the acceptance predicate may affect which candidate is selected but not the acceptance value of a fixed candidate. □

This theorem gives self-description an engineering role. Reflection is not asked to certify itself as truthful by fiat. It supplies checked features to a controller that still submits ordinary certificates to an unchanged verifier.

16 A theorem connecting Level-2 state-awareness to shortest settlement

The half-step result can now be specialized to checked self-state information.

Let

$$s : X \to S$$

be a checker-backed self-description map available to the controller. A Level-2-style $s(x)$ may include the current BANK fingerprint, active role, resource counters, recent verifier outcomes, branch dependencies, and fallback status. Assume these fields are checked against the actual machine state rather than merely asserted by the machine.

Theorem 16.1 (Checked self-state compass sufficiency)**.** *Assume the unit-cost controlled settlement model. Suppose the optimal Abel coordinate factors through checked self-state:*

$$h^*(x) = g(s(x))$$

throughout the relevant basin. Let $\widehat{g}$ satisfy, for every candidate successor y encountered,

$$|\widehat{g}(s(y)) - g(s(y))| < \frac{1}{2}.$$

Then the controller

$$\pi_s(x) \in \arg\max_a \widehat{g}(s(F_a x))$$

selects a shortest-settlement action at every step and reaches settlement in $r^*(x_0)$ steps.

Proof. Set $\widehat{h}(y) = \widehat{g}(s(y))$. By the factorization hypothesis, the error bound is exactly the hypothesis of Theorem 12.1. Apply that theorem at every step. □

The theorem does *not* prove that Level-2 self-description is sufficient in real theorem search. It proves something narrower and more useful: if checked self-state is a sufficient representation for the optimal settlement coordinate and the learned coordinate is accurate enough, then self-aware control recovers optimal next moves. The empirical question is now sharply stated:

Do checked self-state features improve estimation of h^* on held-out searches?

and the ultimate endpoint is

Do they improve verified settlement under fixed resources?

17 Fairness, failure, and the independence route

Adaptive control creates a danger: a controller may learn that some histories correlate with independence and then silently treat prolonged proof failure as a metatheorem. That is not allowed.

Automatic Metalogical Deciders, Part Two makes the canonical rule intentionally conservative: P, R, and I receive equal opportunities under the standard scheduler, and long failure histories are not converted into logical status [8]. Adaptive allocation remains legitimate as an experimental policy provided two boundaries remain clear:

1. only an explicit accepted independence certificate licenses the independence outcome; and
2. any liveness guarantee inherited from complete baseline procedures is protected by a fairness floor or persistent fallback.

This distinction is compatible with learning from failure. A state descriptor may record "P has spent N verifier calls without settlement." Historical data may show that such a state predicts a higher

conditional chance that an I-action will settle soon. The controller may use that information to rank actions. But the statement

"P and R have failed for a long time"

is still not itself the certificate

$$\Gamma \nvdash c \quad \text{and} \quad \Gamma \nvdash \neg c.$$

18 Why negative experiments belong in the theory

A compass theory that only records positive navigation stories is not a scientific control theory. Recent work on compass/density hypotheses supplies useful negative examples. In one frozen DATA 4.1-Q benchmark, a preregistered shell-concentration hypothesis failed, and a strong practical physics-density advantage was denied because the measured mean-rank improvement was only 0.552%, below the preregistered 10% threshold [9]. These outcomes do not show that every physics-inspired or density-based feature is useless. They show that architectural plausibility is not evidence of practical value.

The lesson for the present program is methodological. Theorems should establish safety, invariants, and idealized implications. Experiments should determine whether proposed representations and learned coordinates transfer to new search states. Negative results should eliminate mechanisms, revise hypotheses, or identify missing state variables rather than being hidden behind new terminology.

19 Returning to the business of settling conjectures

The machinery developed above is worthwhile only if it changes how difficult targets are attacked. The next research stage should therefore reduce the proportion of effort devoted to inventing additional architecture and increase the proportion devoted to closed-loop settlement tests.

19.1 A three-stage settlement program

Stage A: solved but hidden targets. Use theorems whose certificates are known to the experimenter but withheld from the search machine. Freeze Γ, target, verifier, candidate rules, resource budget, BANK policy, and random seeds before evaluation. These targets permit retrospective computation of exact or near-exact settlement distances on manageable state graphs and therefore provide labels for r^* or action ordering.

Stage B: mixed sealed settlement instances. Construct a sealed mixture containing theorem, refutation, independence, and inconsistent-base cases with certificate classes declared in advance. Compare unbiased round-robin, standard professional ATP baselines where appropriate, BANKed search, compass-guided search, and reflective compass-guided search. The primary endpoint is time or verifier calls to the first accepted terminal certificate. Timeouts remain UNKNOWN.

Stage C: genuinely difficult mathematics. Only after the controller survives leakage-resistant stages A and B should it be aimed at problems whose status is not known to the system builder. The formal environment must still be frozen. For an open conjecture, a long run that produces no certificate is simply an unsuccessful run. A positive claim requires an independently checkable proof, refutation, model/metacertificate, or inconsistency certificate of the declared kind.

19.2 What to measure

The primary measurement should be end-to-end verified settlement under fixed resources. Useful diagnostics include:

- error or ranking loss for $\widehat{h}$ versus retrospective h^* where computable;
- fraction of states satisfying the local half-step ordering condition;
- P/R/I/C allocation history and fairness-floor usage;
- number and downstream marginal value of BANK deposits;
- reduction in estimated or exact remaining settlement depth after a lemma deposit;
- verifier calls, wall-clock time, peak memory, and restart/checkpoint overhead;
- ablations removing self-state features, FUTUREBANK features, BANK reuse, or compass guidance;
- negative controls that preserve candidate coverage while randomizing ranking.

19.3 The theorem-driven experimental question

The central empirical question can now be stated without metaphor. Let π_0 be a frozen baseline policy and π_R a reflective compass policy using checked state features. On a preregistered target distribution and resource budget B, compare

$$\Pr(\tau_{\text{settle}} \le B \mid \pi_R) \quad \text{with} \quad \Pr(\tau_{\text{settle}} \le B \mid \pi_0),$$

where settlement means an accepted terminal certificate. Secondary questions ask whether gains are mediated by better local action ordering, earlier high-value BANK deposits, or more effective route allocation.

The desired end state is not a machine that has a sophisticated vocabulary for its own search. It is a machine that uses verified memory, counterfactual search, checked self-state, and learned control to produce more independently checkable mathematical settlements.

20 A compact theorem map

Result	Status	Meaning
BANK–SIC representation	Theorem	AMLD BANK evolution becomes a SIC trajectory after forgetting record metadata.
Γ seed correspondence	Corollary	Γ is the admitted initial mathematical state in both SIC and AMLD views.
Proof-Horizon–NSA equivalence	Theorem	Ordinary horizon halting, standard derivability, and proof existence below an unlimited nonstandard cost bound agree on standard targets.
Unlimited-horizon dominance	Corollary	Every standard theorem-specific finite Proof Horizon lies below the unlimited bound K.
Conditional settlement-ATP	Theorem	Fair complete coverage of declared certificate families makes global settlement a target-search ATP for a settlement proposition.
Controlled IFS representation	Theorem	Complete AMLD search evolves by selected state maps F_j.
Full-state noncontractivity	Theorem	Multiple frozen terminal states forbid strict contraction under any metric on the complete unquotiented state.
Quotient contractivization	Theorem	Collapse terminal states and weight depth edges by q^{-r}; the deterministic settling map becomes a contraction.
Hitting-time Abel coordinate	Proposition	$h = -r$ satisfies $h(Tx) = h(x) + 1$ before settlement.
Controlled Abel–Bellman	Theorem	$h^* = -r^*$ obeys $\max_a h^*(F_a x) = h^*(x) + 1$.
Half-step compass	Theorem	Uniform successor error $< 1/2$ recovers shortest-settlement actions in the unit-cost model.
Level-1 self-description capability	Theorem	Effective internalization plus fair proof search lets the tagged global machine prove its own ATP status.
Checked self-state compass sufficiency	Theorem	If checked Level-2-style state is sufficient for h^* and learned within $1/2$, reflective greedy control is optimal.
Transfer to unseen mathematics	Open/empirical	Must be demonstrated by leakage-resistant settlement experiments, not inferred from architecture.

21 Relation to the surrounding research program

The present manuscript is related to, but not dependent on, several larger texts and working directions. *What Checks the Proof?* supplies the verification-centered architecture: Γ, BANK, FUTUREBANK,

P/R/I/C, intrinsic verification, fair fallback, and the distinction between search and certification [2]. Its job is foundational.

Depths of Induction and Formalized Self-Awareness supplies SICs and ATPs as staged formal computers, together with tagging, self-models, self-theories, reflection sequences, and formal self-awareness levels [3]. Its role here is to make the search machine itself an object of formal mathematics.

The *Universal Proof-Horizon Operator* supplies the exhaustive positive-side baseline: under an effectively proper proof cost, a partial computable operator finds a canonical minimum-cost proof exactly on theorem inputs. The NSA chapter of *What Checks the Proof?* supplies a distinct semantic idealization in which an unlimited bound covers every standard finite proof code. Theorem 5.2 connects these formulations while preserving their computational boundary: the nonstandard bound dominates all standard horizons but is not a standard executable cutoff [1, 2].

DATA 3.0 supplies the dynamical backbone: complete state, typed verified BANK, state transformations, controlled/place-dependent iteration, transformation semigroups, target fibers, terminal fixed points, settlement horizons, and the retrospective Abel coordinate [4].

Proof Compass Theory: Learnability, Shrinkage, and Control supplies explicit sufficient conditions under which learned navigation can outperform uninformed ordering while retaining safe fallback; it also records why unrestricted compass learnability is false without visible signal [5]. AMLD compass-control work supplies the optimal-control interpretation: value functions, distance-to-settlement, Lyapunov/Abel coordinates, Bellman feedback, and the operational value of theorem-shell changes [6].

Automatic Metalogical Deciders, Part Two supplies scheduling discipline and negative results about naive resource heuristics, especially the rule that failure is information for control but never a substitute for a certificate [8]. The *Reflective Compass Control Research Program* supplies a nearby implementation research agenda: certified self-state, meta-policy selection, conservative fallback, and native verifier sovereignty [7].

The working title *What Watches the Search?* can naturally develop the reflection/control side further. A specialized working title, *Initial Segments of Tegmark's Self-Aware Substructures: Formal Self-Aware Automated Theorem Provers*, can isolate the philosophical and order-theoretic questions without treating formal self-reference as consciousness. A later *Techniques to Settle Conjectures* can then reverse the emphasis: less architecture, more theorem targets, ablation-tested search strategies, and archived verifier certificates.

The direction of travel should be deliberate:

$$\boxed{\text{build the correctness boundary}} \rightarrow \boxed{\text{understand and control the search}} \rightarrow \boxed{\text{settle mathematics.}}$$

22 Conclusion

The central conceptual bridge is smaller than it first appears. A SIC begins at Γ and materializes a monotone computational state. An AMLD BANK begins at the same Γ, adds verification metadata, and materializes a monotone certified state. After forgetting metadata, the BANK trajectory has the SIC form. Under fair complete coverage of declared certificate classes, the P/R/I/C settlement process becomes an ATP-like target search for a formal settlement proposition. The Proof-Horizon/NSA bridge adds a second axis: theorem-specific ordinary finite horizons remain computable only on theorem inputs, while an unlimited nonstandard bound semantically dominates every standard such horizon without becoming an executable universal runtime.

DATA 3.0 then supplies the correct dynamical language. The machine is a controlled discrete iterated

system, but the full terminal-freezing architecture cannot be a classical contractive Hutchinson IFS when distinct terminal states remain distinct. That negative theorem has a positive companion: quotient the terminal set to one absorbing point and restrict to a deterministic finite-settling basin, and an explicit hitting-time metric makes the induced map contractive. The same remaining-time function produces an exact Abel coordinate, and its controlled optimum produces a Bellman equation that is simultaneously a controlled Abel equation.

The half-step theorem gives the theoretical program a concrete target. A compass that is locally accurate enough need not understand all of mathematics; it must preserve the action ordering that points toward settlement. Checked self-description then has a non-mystical use: it can expose the machine's current BANK, history, resource state, and fallback status as verified control features. If those features are sufficient for the optimal settlement coordinate and can be learned accurately, shortest-settlement control follows under the stated model.

The remaining question is empirical and mathematical rather than terminological: can these ideas make a verifier-gated machine settle harder conjectures than simpler search under the same resource constraints? The appropriate next step is therefore not another layer of metaphor. It is a sequence of sealed, reproducible settlement experiments followed by a return to genuinely difficult conjectures, with every positive claim ending where *What Checks the Proof?* began: at an independently checked certificate.

AI Integrity Statement

AI assistance was used to synthesize earlier manuscripts, test proof ideas, draft exposition, and produce the Version 1.4 typeset manuscript. Mathematical claims labeled as theorems were checked against their stated hypotheses during preparation, but they remain appropriate subjects for independent scholarly review and, where feasible, formal verification. Empirical claims have not been promoted to theorems. No open conjecture is reported as settled without an independently checkable certificate.

References

[1] Brian Tenneson. *The Universal Proof-Horizon Operator: From MIU and the nth Well-Formed Formula to ZFC, Predator, and Automated Logical Deciders*. Version 1.0, August 3, 2026. Live article.

[2] Brian Tenneson. *What Checks the Proof? A Textbook Introduction to Automated Mathematical Logic Deciders: BANK, FUTUREBANK, Imagination, Compass, Verification, Controlled Creativity, and Formalized Self-Awareness*. Textbook Version 6.80, August 22, 2026. Live article.

[3] Brian Tenneson. *Depths of Induction and Formalized Self-Awareness: Formal Systems, Inference Closure, Automated Theorem Proving, Simulations, and Self-Aware Structures*. Version 46/46.1, July 2026.

[4] Brian Tenneson. *DATA 3.0 AMLD IFS Semigroup Architecture*. Working technical manuscript, August 2026.

[5] *Proof Compass Theory: Learnability, Shrinkage, and Control*. Technical report prepared for Brian Tenneson, August 15, 2026. Public project pointer: `https://github.com/btenneson/pub/tree/main/cs.LO_Logic_in_Computer_Science/Proof_Compass_Theory`. Live article.

[6] Brian Tenneson. *AMLD Compass–Control Integration: Value Functions, Lyapunov and Abel Coordinates, Theorem-Shell Control, and Proof-Ocean Navigation*. August 14, 2026.

[7] *Reflective Compass Control Research Program*. Versions 0.1–0.2, August 2026. Working technical research program prepared for Brian Tenneson. Live article.

[8] Brian Tenneson. *Automatic Metalogical Deciders, Part Two: Conjectures Revealed*. Revised edition, August 12, 2026.

[9] Brian Tenneson. *Compass Theorems and Physics-Density Limits*. Version 0.1, August 17, 2026.

[10] Brian Tenneson. *From Search Dynamics to Semi-Ideal Computers: Discovery, Abel Coordinates, and Experimental Design*. Working manuscript/addendum, August 2026. Live article.

[11] Max Tegmark. "Is 'the Theory of Everything' Merely the Ultimate Ensemble Theory?" *Annals of Physics* 270(1):1–51, 1998. DOI: 10.1006/aphy.1998.5855. Preprint: `https://arxiv.org/abs/gr-qc/9704009`.

[12] John E. Hutchinson. "Fractals and Self-Similarity." *Indiana University Mathematics Journal* 30(5):713–747, 1981. DOI: 10.1512/iumj.1981.30.30055.

[13] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[14] John Harrison. *Handbook of Practical Logic and Automated Reasoning*. Cambridge University Press, 2009.

[15] Norman D. Megill and David A. Wheeler. *Metamath: A Computer Language for Mathematical Proofs*. Second edition, 2019. `https://us.metamath.org/downloads/metamath.pdf`.